

計算ゼミ 課題 3

石川曹太

2026 年 7 月 8 日

1 コード

Listing 1 メインプログラム

```
1 program ex03
2   use, intrinsic :: iso_fortran_env
3   use mod_linear_solver_lu
4
5   implicit none
6
7   real(real64), allocatable :: A(:, :), Ainv(:, :)
8
9   integer :: i
10
11  !使い分け
12  !integer :: n = 3
13  integer :: n = 2
14
15  allocate (A(n, n))
16  allocate (Ainv(n, n))
17
18  !使い分け
19  !A = reshape([1.0d0, -2.0d0, 0.0d0, 1.0d0, 0.0d0, 2.0d0, -1.0d0, 1.0d0, 1.0
20    d0], [n, n])
21  !A = reshape([0.0d0, 2.0d0, 1.0d0, 1.0d0, 3.0d0, 0.0d0, 0.0d0, 1.0d0, 2.0d0
22    ], [n, n])
23  A = reshape([0.0d0, 2.0d0, 1.0d0, 3.0d0], [n, n])
24
25  print *, "A ="
26  do i = 1, n
27    write(*, '(*(F25.15,1X))') A(i, :)
28  end do
29
30  call matrix_inverse(A, Ainv)
31
32  print *, "Ainv ="
```

```

32     do i = 1, n
33         write(*, '(*(F25.15,1X))') Ainv(i, :)
34     end do
35
36     deallocate (A, Ainv)
37
38
39 end program ex03

```

Listing 2 線形ソルバーモジュール

```

1 module mod_linear_solver_lu
2
3     use ,intrinsic :: iso_fortran_env
4     implicit none
5     private
6     public :: matrix_inverse
7
8 contains
9
10 !-----
11 ! 逆行列計算
12 !-----
13     subroutine matrix_inverse(A, Ainv)
14
15         real(real64), intent(in) :: A(:, :)
16         real(real64), intent(out) :: Ainv(:, :)
17
18         real(real64), allocatable :: P(:, :), L(:, :), U(:, :), Linv(:, :),
19             Uinv(:, :)
20         integer :: n, m
21
22         n = size(A, 1)
23         m = size(A, 2)
24
25         !それぞれの行列にサイズを決定
26         allocate(P(n, m), L(n, m), U(n, m), Linv(n, m), Uinv(n, m))
27
28         call LUdecomposition(P, A, L, U)
29         call Linverse(L, Linv)
30         call Uinverse(U, Uinv)
31
32         !Ainv = Uinv*Lin*P
33         Ainv = matmul(Uinv, matmul(Linv, P))
34
35         !数値計算ではallocateしたらdeallocateするのが習慣らしい
36         deallocate(P, L, U, Linv, Uinv)
37
38     end subroutine

```

```

38
39
40 ! -----
41 ! LU分解
42 ! -----
43
44     subroutine LUdecomposition(P, A, L, U)
45
46         real(real64), intent(in) :: A(:, :)
47         real(real64), intent(out) :: P(:, :), L(:, :), U(:, :)
48
49         ! nは行列のサイズ, i はPとLの初期値として1を導入する, kはピボット操作の回数
50         integer :: n, i, k
51
52         n = size(A, 1)
53
54         ! Aは固定し, Uを動かして最終的に  $PA = LU$  を目指す
55         U = A
56
57         ! PとLは最初に全成分を0にしてから i を用いて対角成分に1を挿入
58         P = 0.0d0
59         L = 0.0d0
60         do i = 1, n
61             P(i, i) = 1.0d0
62             L(i, i) = 1.0d0
63         end do
64
65         do k = 1, n-1
66
67             call Pivot(P, L, U, k)
68             call GaussElimination(L, U, k)
69
70         end do
71
72     end subroutine LUdecomposition
73
74
75 ! -----
76 ! ピボット選択
77 ! -----
78
79     subroutine Pivot(P, L, U, k)
80
81         real(real64), intent(inout) :: P(:, :), L(:, :), U(:, :)
82         integer, intent(in) :: k
83
84         ! nは行列Uのサイズ, i はピボットとなる最大絶対値を探索するループで使う, pivot_rowは最終的にk行と入れ替える行
85         integer :: n, i, pivot_row

```

```

86
87      !maxは探索する中での最大値, Ozは入れ替えを行う際基準となるk行を1行分丸ごと保存
88      real(real64) :: max, Pz(size(P, 2)), Lz(size(L, 2)), Uz(size(U, 2))
89
90      n = size(U, 1)
91      pivot_row = k
92      max = abs(U(k, k))
93
94      do i = k+1, n
95
96          if (abs(U(i, k)) > max) then
97              max = abs(U(i, k))
98              pivot_row = i
99          end if
100
101      end do
102
103      !Uのピボットを見つけたら合わせてP, L, Uの成分を入れ替える
104      !ただし, Lに関しては確定した対角成分以外を入れ替えるため発動する条件を付与する
105
106      if (pivot_row /= k) then
107
108          Pz = P(pivot_row, :)
109          P(pivot_row, :) = P(k, :)
110          P(k, :) = Pz
111
112          Uz = U(pivot_row, :)
113          U(pivot_row, :) = U(k, :)
114          U(k, :) = Uz
115
116      end if
117
118      if (k > 1) then
119
120          Lz(1:k-1) = L(pivot_row, 1:k-1)
121          L(pivot_row, 1:k-1) = L(k, 1:k-1)
122          L(k, 1:k-1) = Lz(1:k-1)
123
124      end if
125
126      end subroutine Pivot
127
128
129      !-----
130      ! ガウス消去
131      !-----
132      subroutine GaussElimination(L, U, k)
133
134      real(real64), intent(inout) :: L(:, :), U(:, :)

```

```

135     integer, intent(in) :: k
136
137     ! nは行列のサイズ, 行列Uの成分をU ( i , j )とし, それぞれの成分で計算を行う
138     integer :: n, m, i, j
139     ! factorはピボット選択後ガウス消去を行うために行以外の行に掛ける係数k
140     real(real64) :: factor
141
142     n = size(U, 1)
143     m = size(U, 2)
144
145     do i = k+1, n
146
147         factor = U(i, k) / U(k, k)
148         ! factorが決定すると行列Lの下三角の成分が決定する
149         L(i, k) = factor
150
151         do j = 1, m
152
153             U(i, j) = U(i, j) - U(k, j) * factor
154
155         end do
156
157     end do
158
159     end subroutine GaussElimination
160
161
162     ! -----
163     ! Lの逆行列計算 (下三角) 前進代入
164     ! -----
165     subroutine Linverse(L, Linv)
166
167         real(real64), intent(in) :: L(:, :)
168         real(real64), intent(out) :: Linv(:, :)
169
170         ! nは行列Lのサイズ, Lの成分をL ( i , j )とする
171         integer :: n, m, i, j, k
172         ! s (sum) は一時的に和を保存して, 最終的に - L ( j , j ) で割る 数値計算でよく用いる手法
173         real(real64) :: s
174
175         n = size(L, 1)
176         m = size(L, 2)
177
178         ! L i n vの中身は後から挿入するとして, 一旦全成分を0で統一
179         Linv = 0.0d0
180
181         ! 1列目から考える
182         do j = 1, m
183

```

```

184         Linv(j, j) = 1.0d0 / L(j, j)
185
186         !上では対角成分を確定させたから、対角以下の成分を考える
187         do i = j+1, n
188
189             s = 0.0d0
190
191             do k = 1, i-1
192
193                 s = s + L(i, k) * Linv(k, j)
194
195             end do
196
197             Linv(i, j) = -s / L(i, i)
198
199         end do
200
201     end do
202
203     end subroutine Linverse
204
205
206     !-----
207     ! Uの逆行列計算（上三角）後退代入
208     !-----
209     subroutine Uinverse(U, Uinv)
210
211         real(real64), intent(in) :: U(:, :)
212         real(real64), intent(out) :: Uinv(:, :)
213
214         integer :: n, m, i, j, k
215         real(real64) :: s
216
217         n = size(U, 1)
218         m = size(U, 2)
219
220         Uinv = 0.0d0
221
222         !後退代入なので i, j は後ろから考える
223
224         do j = m, 1, -1
225
226             Uinv(j, j) = 1.0d0 / U(j, j)
227
228             do i = j-1, 1, -1
229
230                 s = 0.0d0
231
232                 do k = i+1, j

```

```

233
234         s = s + U(i, k) * Uinv(k, j)
235
236     end do
237
238     Uinv(i, j) = -s / U(i, i)
239
240 end do
241
242 end do
243
244 end subroutine Uinverse
245
246
247 end module mod_linear_solver_lu

```

2 LU 分解の説明

2.1 LU 分解とは

LU 分解とは、ある正方行列 A を下三角行列 L と上三角行列 U の積に分解する手法である。これは数値解析において最も基本的な行列分解の一つであり、連立一次方程式の求解や逆行列の計算など、多くの数値計算で利用されている。

$$A = LU \quad (1)$$

下三角行列とは行列の下半分の三角部分にのみ非ゼロの要素が存在し、上半分の三角部分は全てゼロであるという行列である。一方で上三角行列は逆に、上半分の三角部分のみ非ゼロで、下半分はゼロであるという行列のことである。

$$L = \begin{bmatrix} 1 & 0 & 0 & \dots & 0 \\ L_{21} & 1 & 0 & \dots & 0 \\ L_{31} & L_{32} & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & 0 \\ L_{n1} & L_{n2} & L_{n3} & \dots & 1 \end{bmatrix} \quad (2)$$

$$U = \begin{bmatrix} U_{11} & U_{12} & U_{13} & \dots & U_{1n} \\ 0 & U_{22} & U_{23} & \dots & U_{2n} \\ 0 & 0 & U_{33} & \dots & U_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & 0 & U_{nn} \end{bmatrix} \quad (3)$$

ここでは各行列を一意的に決めるために、今回のレポートでは行列 L の対角成分は全て 1 としている。

LU 分解を用いると、一度分解を行うだけで複数の連立一次方程式を効率よく解くことができるため、大規模な数値計算では非常に重要な手法になっている。

2.2 部分ピボット選択

実際の数値計算では、単純な LU 分解だけでは計算が失敗する場合が存在する。例として、次に示すような行列の LU 分解を考える。

$$A = \begin{bmatrix} 0 & 3 & 1 \\ 1 & -2 & 3 \\ -2 & 1 & 4 \end{bmatrix}$$

後の 2.3 節で示すガウス消去法では行列の対角成分で割るという操作が入るため、この行列の A_{11} 成分で割ろうとするとゼロ割になって計算ができなくなる。また、対象となる対角要素がゼロでなくとも、他の要素に比べて非常に小さい場合、割り算をすることによって消去法における係数が大きくなってしまい桁落ちなどの数値誤差が生じやすくなってしまいうというデメリットがある。そのため例に示した行列の場合、1 行目と他の行を交換した行列を考え LU 分解を進める必要がある。この時第 k 列について $|A_{ik}|$ が最大となる行を探索し、その行を第 k 行と交換してからガウス消去を行う。今回の場合、 A_{31} 要素の -2 が一番大きいため、3 行目と 1 行目を交換する。

$$\begin{bmatrix} -2 & 1 & 4 \\ 1 & -2 & 3 \\ 0 & 3 & 1 \end{bmatrix}$$

このような行列の交換操作をピボット選択という。今回の交換のような行列の行のみを入れ替えることを部分ピボット選択 (partial pivoting)、行も列も入れ替える操作を完全ピボット選択 (full pivoting) という。通常のガウス消去法のみを用いた LU 分解とは違い、部分ピボット選択付き LU 分解では最終的に行を交換し終えた行列に対して行われる。この行列の行を交換するための変換行列 P を用いると、

$$PA = LU \tag{4}$$

が成立する。ここでの変換行列 P は置換行列と呼ばれ、次のように単位行列を行列 A の入れ替えたい行同士と同じ行で入れ替えた形をしている。

$$P = \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix}$$

置換行列は行の交換を表す行列であり、ピボット探索を行い行交換をするたびに更新される。また、すでに求まっている L の要素にも対応する行を交換する必要があるため、本プログラムでは P, U , と L の確定済み部分を同時に交換するものとしている。

本プログラムの部分ピボット選択では、 $1 \sim n-1$ (n は対象とする正方行列の行数) という範囲の k でピボット探索をし、それぞれの探索後に P, U, L の行交換を行うという操作をループさせている。具体的には U_{kk} の数値を $\max$ の初期値として設定し、 k 列の他の成分を U_{ik} の i ループ ($k+1 \sim n$) を回して比較することにより、最終的な $\max$ と、1 行目と交換する行の行番号を `pivot.row` として保存する。

その後、 $\max$ の存在する行をそれぞれ Pz, Uz, Lz に格納し、 k 行目との交換を行う。ただ、 L に関しては確定した対角成分以外を入れ替えるため、行交換が発動する条件 $k > 1$ に設定し、該当箇所のみを Lz に格納するものとしている。

2.3 ガウス消去

ピボットが決定すると、ガウス消去によって対角成分より下の部分を順番にゼロにしていく操作を行う。行列 U の成分を U_{ij} とし、それぞれループを回すことで全体の成分の計算を行う。ここでガウス消去を行うために k 行以外の行に掛ける係数を factor として定義し、以下の式で得る。ここで得た factor の値は自動的に L の下三角成分に格納される。

$$\text{factor} = \frac{U_{ik}}{U_{kk}} = l_{ik} \quad (5)$$

その後、各行各列に以下の計算を行うことで対象とする k 列の対角成分の下部分がゼロになった行列を得ることができる。

$$U_{ij} = U_{ij} - U_{kj}l_{ik} \quad (6)$$

本プログラムではこれまでに説明したピボット選択とガウス消去をそれぞれサブルーチンとして作成し、LU 分解という一つのサブルーチンにまとめて格納したうえで k のループを回している。この二つの操作の繰り返しによって最終的に $PA = LU$ を満たすような行列 L, U が得られるコードになっている。

2.4 下三角行列の逆行列

LU 分解の終了後、まず下三角行列 L の逆行列を求める。その逆行列を一旦 X と置くと、以下の関係を満たす。

$$LX = I \quad (7)$$

$$\begin{bmatrix} L_{11} & 0 & 0 \\ L_{21} & L_{22} & 0 \\ L_{31} & L_{32} & L_{33} \end{bmatrix} \begin{bmatrix} X_{11} & X_{12} & X_{13} \\ X_{21} & X_{22} & X_{23} \\ X_{31} & X_{32} & X_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (8)$$

この等式を成分で書き下すと、

$$\begin{cases} L_{11}X_{11} = 1 \\ L_{21}X_{11} + L_{22}X_{21} = 0 \\ L_{31}X_{11} + L_{32}X_{21} + L_{33}X_{31} = 0 \end{cases} \begin{cases} L_{11}X_{12} = 0 \\ L_{21}X_{12} + L_{22}X_{22} = 1 \\ L_{31}X_{12} + L_{32}X_{22} + L_{33}X_{32} = 0 \end{cases} \begin{cases} L_{11}X_{13} = 0 \\ L_{21}X_{13} + L_{22}X_{23} = 0 \\ L_{31}X_{13} + L_{32}X_{23} + L_{33}X_{33} = 1 \end{cases} \quad (9)$$

となる．ここで， L の成分は確定しているから，一番上の式から X の値を求めて順次代入していけば簡単にすべての未知数 X を求めることができる．この計算方法を前進代入という．行列 X の対角より上の成分は U_{11} がゼロでないことを考慮するとゼロであると求められる．

次に対角成分は，以下の式で計算可能である．

$$X_{jj} = \frac{1}{L_{jj}} \quad (10)$$

最後に，対角より下の成分は式 (9) より，以下のように求められる．

$$X_{ij} = -\frac{\sum_{k=1}^{i-1} L_{ik}X_{kj}}{L_{ii}} \quad (11)$$

今回は，列ごとに切り分けて考えるため，列を表す j を $1 \sim m$ (m は行列の列サイズ) でループさせて，その中で考慮する対角以下の成分を，行を表す i を用いて $j+1 \sim n$ (n は行列の行サイズ) でループさせる．またそれぞれの成分ごとの積の和を保存するために s という変数を導入し，随時更新していくことで式 (11) の分子を確定させる．

以上の操作によって計算した逆行列を Linv として出力する．

2.5 上三角行列の逆行列

次に，上三角行列 U の逆行列を求める．ここでは，その逆行列を Y と置くと， L の時と同様に以下の関係が得られる．

$$UY = I \quad (12)$$

$$\begin{bmatrix} U_{11} & U_{12} & U_{13} \\ 0 & U_{22} & U_{23} \\ 0 & 0 & U_{33} \end{bmatrix} \begin{bmatrix} Y_{11} & Y_{12} & Y_{13} \\ Y_{21} & Y_{22} & Y_{23} \\ Y_{31} & Y_{32} & Y_{33} \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (13)$$

この等式を成分で書き下すと，

$$\begin{cases} U_{11}Y_{11} + U_{12}Y_{21} + U_{13}Y_{31} = 1 \\ U_{22}Y_{21} + U_{23}Y_{31} = 0 \\ U_{33}Y_{31} = 0 \end{cases} \begin{cases} U_{11}Y_{12} + U_{12}Y_{22} + U_{13}Y_{32} = 0 \\ U_{22}Y_{22} + U_{23}Y_{32} = 1 \\ U_{33}Y_{32} = 0 \end{cases} \begin{cases} U_{11}Y_{13} + U_{12}Y_{23} + U_{13}Y_{33} = 0 \\ U_{22}Y_{23} + U_{23}Y_{33} = 0 \\ U_{33}Y_{33} = 1 \end{cases} \quad (14)$$

となる．ここで、 U の成分は確定しているから、下三角行列の時とは反対に、一番下の式から Y の値を求めて順次代入していけば簡単にすべての未知数 Y を求めることができる．この計算方法を後退代入という．行列 Y の対角より下の成分は U_{11} がゼロでないことを考慮するとゼロであると求められる．

次に対角成分は、以下の式で計算可能である．

$$Y_{jj} = \frac{1}{U_{jj}} \quad (15)$$

最後に、対角より上の成分は式 (14) より、以下のように求められる．

$$Y_{ij} = -\frac{\sum_{k=i+1}^j U_{ik} Y_{kj}}{U_{ii}} \quad (16)$$

今回も下三角行列と同様に、列ごとに切り分けて考える．ここで、今回の場合後退代入を行うため、前節で踏んだステップを 3 列目から考える必要がある．そのため、列を表す j を $m \sim 1$ (m は行列の列サイズ) の順番でループさせて、その中で考慮する対角以上の成分を、行を表す i を用いて $j-1 \sim 1$ でループさせる．また今回もそれぞれの成分ごとの積の和を保存するために s という変数を導入し、随時更新していくことで式 (16) の分子を確定させる．

以上の操作によって計算した逆行列を U_{inv} として出力する．

2.6 逆行列の計算

これまで求めた、 L_{inv} や U_{inv} を用いて、最後に行列 A の逆行列を計算する．LU 分解より式 (4) が成り立つ．両辺の逆をとり、 A の逆行列について解くと、等式を以下のように変形することができる．

$$\begin{aligned} (PA)^{-1} &= (LU)^{-1} \\ A^{-1}P^{-1} &= U^{-1}L^{-1} \\ A^{-1} &= U^{-1}L^{-1}P \end{aligned} \quad (17)$$

今回のプログラムでは、最終的な L と U の逆行列、また行交換の情報を保持した置換行列をそのまま利用できる形で保存している．従って、逆行列計算のサブルーチンでは LU 分解や逆行列計算を行うサブルーチンを用いて必要な行列の情報を得たうえで式 (17) の最後の式を計算することで、元の行列 A の逆行列を A_{inv} として出力している．

3 テストケースの確認

3 つのケースに対して作成したプログラムが通った結果を以下に示す．

```

● ishikawa@ishikawa:~/Desktop/keisanki_semi/programing_ex/ex03$ gfortran ex03.f90 mod_linear_solver_lu.f90 -o ex03.o
● ishikawa@ishikawa:~/Desktop/keisanki_semi/programing_ex/ex03$ ./ex03.o
A =
  1.0000000000000000    1.0000000000000000   -1.0000000000000000
 -2.0000000000000000    0.0000000000000000    1.0000000000000000
  0.0000000000000000    2.0000000000000000    1.0000000000000000
Ainv =
 -0.5000000000000000   -0.7500000000000000    0.2500000000000000
  0.5000000000000000    0.2500000000000000    0.2500000000000000
 -1.0000000000000000   -0.5000000000000000    0.5000000000000000

```

図1 case1 の実行結果

```

● ishikawa@ishikawa:~/Desktop/keisanki_semi/programing_ex/ex03$ gfortran ex03.f90 mod_linear_solver_lu.f90 -o ex03.o
● ishikawa@ishikawa:~/Desktop/keisanki_semi/programing_ex/ex03$ ./ex03.o
A =
  0.0000000000000000    1.0000000000000000
  2.0000000000000000    3.0000000000000000
Ainv =
 -1.5000000000000000    0.5000000000000000
  1.0000000000000000    0.0000000000000000

```

図2 case2 の実行結果

```

● ishikawa@ishikawa:~/Desktop/keisanki_semi/programing_ex/ex03$ gfortran ex03.f90 mod_linear_solver_lu.f90 -o ex03.o
● ishikawa@ishikawa:~/Desktop/keisanki_semi/programing_ex/ex03$ ./ex03.o
A =
  0.0000000000000000    1.0000000000000000    0.0000000000000000
  2.0000000000000000    3.0000000000000000    1.0000000000000000
  1.0000000000000000    0.0000000000000000    2.0000000000000000
Ainv =
 -2.0000000000000000    0.6666666666666667   -0.3333333333333333
  1.0000000000000000    0.0000000000000000    0.0000000000000000
  1.0000000000000000   -0.3333333333333333    0.6666666666666667

```

図3 case3 の実行結果